CASE STUDY 3: BANK ACCOUNT AND MONEY TRANSFERS

Problem Statement

A retail banking organization wants to develop a Java-based banking application that allows customers to maintain accounts, deposit money, withdraw money, and transfer funds between accounts.

Recently, the bank has experienced several operational problems:

- Customers are withdrawing amounts beyond their permitted overdraft limits.
- Money is deducted from one account but is not credited to the destination account when a system failure occurs during a transfer.
- Frozen accounts are still accepting withdrawal requests.
- Account balances do not always match the corresponding transaction history.
- Some transactions are processed with zero or negative amounts.

The development team has proposed a basic implementation using public fields, primitive data types, and separate service methods.

Your team is required to analyse these problems and propose an improved object-oriented domain model for the banking application.

Questions

Q1. Identification of Domain Invariants (6 Marks)

Identify at least six business invariants that must be maintained in the banking application.

For each invariant:

- State the business rule clearly.
- Explain why the rule is important.
- Identify the possible consequences if the rule is violated.

Q2. Entities and Value Objects (6 Marks)

Identify the important domain concepts involved in the banking application.

Classify the following concepts as entities or value objects:

- Bank Account
- Account Number
- Money
- Ledger Entry

- Overdraft Limit
- Fund Transfer

Justify your classification based on identity, equality, lifecycle and mutability.

Q3. Self-Enforcing Objects and Java Design (8 Marks)

Consider the following operations:

- Deposit money.
- Withdraw money.
- Freeze an account.
- Unfreeze an account.
- Close an account.

Design suitable Java classes and methods to implement these operations.

Explain how your design ensures that:

- Invalid transaction amounts are rejected.
- Accounts cannot be withdrawn from when they are frozen or closed.
- The overdraft limit is respected.
- Account data cannot be modified directly by external classes.

Q4. Money Transfer and Aggregate Consistency (6 Marks)

Consider a situation where a customer transfers ₹5,000 from Account A to Account B.

During the transfer, the system successfully deducts the amount from Account A but crashes before crediting Account B.

Answer the following:

1. What problems can arise from this situation?
2. Which business invariants are violated?
3. How would you design the transfer operation to prevent money loss?
4. How would your design handle failures during the transfer?
5. How would your approach differ if the two accounts belonged to different banking systems?

Q5. Ledger Management and Design Evaluation (4 Marks)

The bank wants to maintain a complete transaction history for every account.

1. What information should a ledger entry contain?
2. Should an existing ledger entry be modified or deleted after a transaction is completed? Justify your answer.
3. How would you ensure that the account balance remains consistent with the transaction history?
4. What challenges might arise when the transaction history becomes very large?

Code:

```
import java.math.BigDecimal;

enum AccountStatus {
    ACTIVE, FROZEN, CLOSED
}

class Money {
    private final BigDecimal amount;

    public Money(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0) {
            throw new IllegalArgumentException(
                "Amount must be positive");
        }
        this.amount = amount;
    }

    public BigDecimal getAmount() {
        return amount;
    }
}

class BankAccount {
    private final String accountNumber;
    private BigDecimal balance;
    private final BigDecimal overdraftLimit;
    private AccountStatus status;

    public BankAccount(String accountNumber,
                       BigDecimal balance,
                       BigDecimal overdraftLimit) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.overdraftLimit = overdraftLimit;
        this.status = AccountStatus.ACTIVE;
    }

    public void deposit(Money money) {
```

```java
class BankAccount {
    private final String accountNumber;
    private BigDecimal balance;
    private final BigDecimal overdraftLimit;
    private AccountStatus status;

    public BankAccount(String accountNumber,
                       BigDecimal balance,
                       BigDecimal overdraftLimit) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.overdraftLimit = overdraftLimit;
        this.status = AccountStatus.ACTIVE;
    }

    public void deposit(Money money) {
        checkActive();
        balance = balance.add(money.getAmount());
    }

    public void withdraw(Money money) {
        checkActive();

        BigDecimal newBalance =
            balance.subtract(money.getAmount());

        if (newBalance.compareTo(overdraftLimit.negate()) < 0) {
            throw new IllegalArgumentException(
                "Overdraft limit exceeded");
        }

        balance = newBalance;
    }
```

```java
public void freeze() {
    if (status == AccountStatus.CLOSED)
        throw new IllegalStateException("Account closed");

    status = AccountStatus.FROZEN;
}

public void unfreeze() {
    if (status != AccountStatus.FROZEN)
        throw new IllegalStateException("Not frozen");

    status = AccountStatus.ACTIVE;
}

public void close() {
    if (balance.signum() != 0)
        throw new IllegalStateException(
            "Balance must be zero to close");

    status = AccountStatus.CLOSED;
}

private void checkActive() {
    if (status != AccountStatus.ACTIVE)
        throw new IllegalStateException(
            "Account is not active");
}

public BigDecimal getBalance() {
    return balance;
}

public String getAccountNumber() {
    return accountNumber;
}
```

```java
    }
}
public class BankingApp {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {

        BankAccount a = new BankAccount(
            accountNumber: "1001",
            new java.math.BigDecimal("10000"),
            new java.math.BigDecimal("5000")
        );

        BankAccount b = new BankAccount(
            accountNumber: "1002",
            new java.math.BigDecimal("2000"),
            new java.math.BigDecimal("5000")
        );

        Money amount = new Money(
            new java.math.BigDecimal("500")
        );

        a.withdraw(amount);
        b.deposit(amount);

        System.out.println("Account A: " + a.getBalance());
        System.out.println("Account B: " + b.getBalance());
    }
}
```

Output:

```
[Done] exited with code=1 in 0.679 seconds

[Running] cd "c:\Users\manoj kumar\Desktop\2ND YEAR SEM 2\AOOP\day2.java\" && javac BankingApp.java && java BankingApp
Account A: 9500
Account B: 2500

[Done] exited with code=0 in 0.759 seconds
```